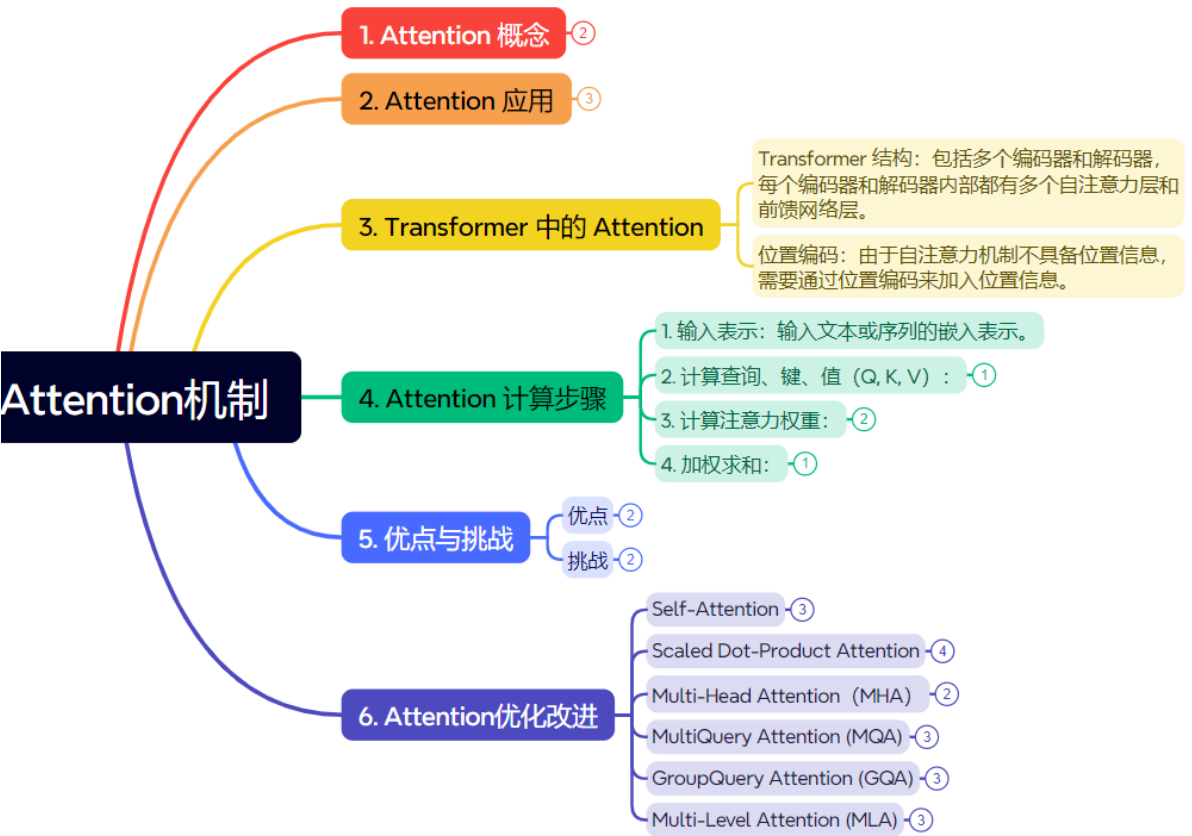


引言

在深度学习，特别是自然语言处理（NLP）领域，注意力机制（Attention Mechanism）是一个非常重要的概念。Attention机制的起源可以追溯到对生物视觉注意力的模拟以及神经机器翻译的实际需求。Bahdanau等人的工作首次将Attention机制引入自然语言处理领域；而Transformer架构则将Attention机制推向了一个新的高度，使其成为现代自然语言处理的核心技术之一。随着DeepSeek的爆火，它们的MLA注意力方法更是将Attention机制的应用和优化发展到了极致

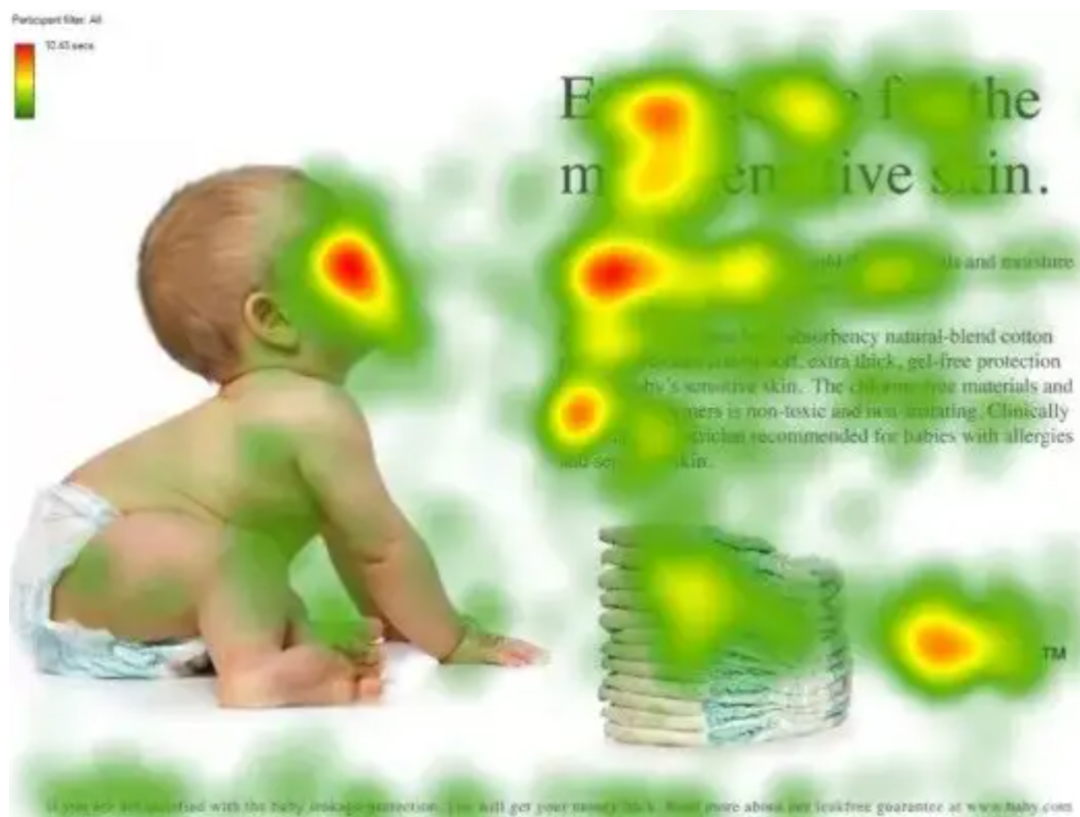
Attention机制核心思想是在处理数据时，模型可以有选择性地关注输入的不同部分，进而提升模型的性能。目前，它已经出现了多个升级优化版本，MHA（Mutil Head Attention，多头注意力）、MQA（Mutil Query Attention，多请求注意力）、GQA（Group Query Attention，组请求注意力）、MLA（Multi-Head Latent Attention，多头潜注意力）等。本文将详细介绍这些Attention变体及其实现方法，让你一次性了解当前主流的注意力机制。



视觉注意力

为了方便大家理解，这里简单介绍一下视觉的选择性注意力机制，让大家从主观上有一个大概的理解。视觉注意力机制是人类视觉所特有的大脑信号处理机制。人类视觉通过快速扫描全局图像，获得需要重点关注的目标区域，也就是一般所说的注意力焦点，而后对这一区域投入更多注意力资源，以获取更多所需要关注目标的细节信息，而抑制其他无用信息。

这是人类利用有限的注意力资源从大量信息中快速筛选出高价值信息的手段，是人类在长期进化中形成的一种生存机制，人类视觉注意力机制极大地提高了视觉信息处理的效率与准确性。



上图中展示了人类在看到一副图像时是如何高效分配有限的注意力资源的，其中红色区域表明视觉系统更关注的目标，很明显对于图中所示的场景，人们会把注意力更多投入到人的脸部，文本的标题以及文章首句等位置。

深度学习中的注意力机制从本质上讲和人类的选择性视觉注意力机制类似，核心目标也是从众多信息中选择出对当前任务目标更关键的信息。

在自然语言处理领域，**Attention机制最早是为了改善神经机器翻译（NMT）的效果而引入的**。传统的神经机器翻译模型基于编码器-解码器架构，使用循环神经网络（RNN）处理序列数据。然而，RNN存在“遗忘”问题，并且在解码过程中无法明确地对齐源语言和目标语言的单词。为此，2014年，Bahdanau等人引入了注意力机制来解决传统神经机器翻译模型的局限性。具体来说，他们在解码过程中为每个输出单词动态计算输入序列中每个单词的重要性权重，从而生成上下文向量，用于生成当前单词。这种方法使模型能够更好地对齐源语言和目标语言的单词，显著提高了翻译质量。

随着2017年Google Brain团队论文《Attention Is All You Need》的爆火，**Attention机制在自然语言处理中的重要性得到了进一步提升**，论文提出了完全基于Attention机制的Transformer架构，彻底摒弃了传统的循环神经网络结构。Transformer通过自注意力（Self-Attention）机制同时处理序列中的所有单词，并计算它们之间的关系权重，从而能够更高效地处理长距离依赖关系。

Transformer架构的出现可以说是全球人工智能快速发展的转折点，该架构由Encoder和Decoder两部分组成，其中Encoder部分发展成了Bert、Roberta等模型，Decoder部分发展成了GPT等生成式大模型，毫不客气的说，当前我们熟知的生成大模型的模型架构基本上全部都基于Decoder构建的。此类模型效果强悍，并得到了广泛的应用，这进一步推动了Attention机制的发展。

传统注意力机制

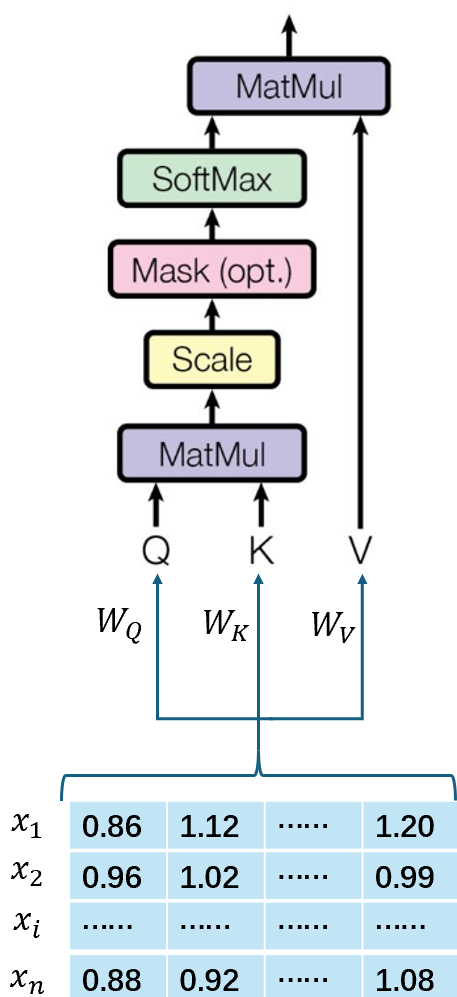
单头注意力只使用一个注意力头来计算权重，从而降低计算复杂度，同时保留注意力机制的核心思想。

原理介绍

对于一个输入序列中的某个词，都会与序列中的所有词计算相关性。假设有一个输入序列：对于每个词，我们计算它与所有其他词的相关性，并赋予不同的权重，然后将这些信息进行加权求和，得到新的表示。当前这里的每个词都要在经过Embedding之后，再做权重转换。下面把最经典的Attention计算公式放在这里：

$$\text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) V$$

公式对应的流程图如下图所示:



按照上图，这里把Attention计算分成以下几个步骤：

(1) 计算 Query, Key, Value 矩阵

每个输入词都会被映射成三个不同的向量：

- 是查询（Query），其表示当前需要关注的内容，例如在机器翻译中，查询可能是目标语言句子中的一个词。
- 是键（Key），表示与查询进行匹配的内容，例如源语言句子中的词。
- 是值（Value），表示最终要提取的信息，通常与键对应。

定义转换矩阵：

其中， W_Q 是可学习的参数矩阵。

(2) 计算点积

计算查询 和键 的点积，得到注意力分数矩阵：

(3)缩放：将点积结果除以 $\sqrt{d_k}$ ：其中， d_k 是 Key 向量的维度，作为缩放因子，避免数值过大导致梯度消失问题。

(4)softmax归一化：对缩放后的点积结果应用softmax函数，得到注意力权重矩阵

(5)加权求和：将注意力权重矩阵与值 相乘，得到加权求和的结果

示例代码

单头注意力机制代码实现（可直接运行）

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class SingleHeadAttention(nn.Module):
    def __init__(self, embed_dim):
        """
        单头注意力机制的初始化。
        :param embed_dim: 嵌入维度，Query、Key 和 Value 的维度
        """
        super(SingleHeadAttention, self).__init__()
        self.embed_dim = embed_dim

        # 定义线性层，将输入映射到 Query、Key 和 Value
        self.query_linear = nn.Linear(embed_dim, embed_dim)
```

```

self.key_linear = nn.Linear(embed_dim, embed_dim)
self.value_linear = nn.Linear(embed_dim, embed_dim)

# 缩放因子，用于防止点积结果过大
self.scale = torch.sqrt(torch.FloatTensor([embed_dim]))

def forward(self, query, key, value):
    """
    单头注意力的前向传播。
    :param query: 查询张量，形状为 $batch\_size, seq\_len\_q, embed\_dim$
    :param key: 键张量，形状为 $batch\_size, seq\_len\_k, embed\_dim$
    :param value: 值张量，形状为 $batch\_size, seq\_len\_k, embed\_dim$
    :return: 输出张量，形状为 $batch\_size, seq\_len\_q, embed\_dim$
    """

    # 将输入映射到 Query、Key 和 Value
    Q = self.query_linear(query)
    K = self.key_linear(key)
    V = self.value_linear(value)

    # 计算点积注意力分数
    attention_scores = torch.matmul(Q, K.transpose(-2, -1)) / self.scale

    # 应用 Softmax 函数，得到注意力权重
    attention_weights = F.softmax(attention_scores, dim=-1)

    # 加权求和，得到最终输出
    output = torch.matmul(attention_weights, V)

    return output, attention_weights

# 示例输入
# 假设我们有以下输入张量：
# - query: $batch\_size, seq\_len\_q, embed\_dim$
# - key: $batch\_size, seq\_len\_k, embed\_dim$
# - value: $batch\_size, seq\_len\_k, embed\_dim$
batch_size = 2
seq_len_q = 3# query的序列长度
seq_len_k = 4#k,v的序列长度，注意这里K、V是成对存在的
embed_dim = 6# 假设embedding的维度为6

# 随机生成输入数据
query = torch.randn(batch_size, seq_len_q, embed_dim)
key = torch.randn(batch_size, seq_len_k, embed_dim)
value = torch.randn(batch_size, seq_len_k, embed_dim)

# 初始化单头注意力模块
attention = SingleHeadAttention(embed_dim)

# 前向传播
output, attention_weights = attention(query, key, value)

# 打印输出
print("Query:\n", query)
print("Key:\n", key)
print("Value:\n", value)
print("Output:\n", output)

```

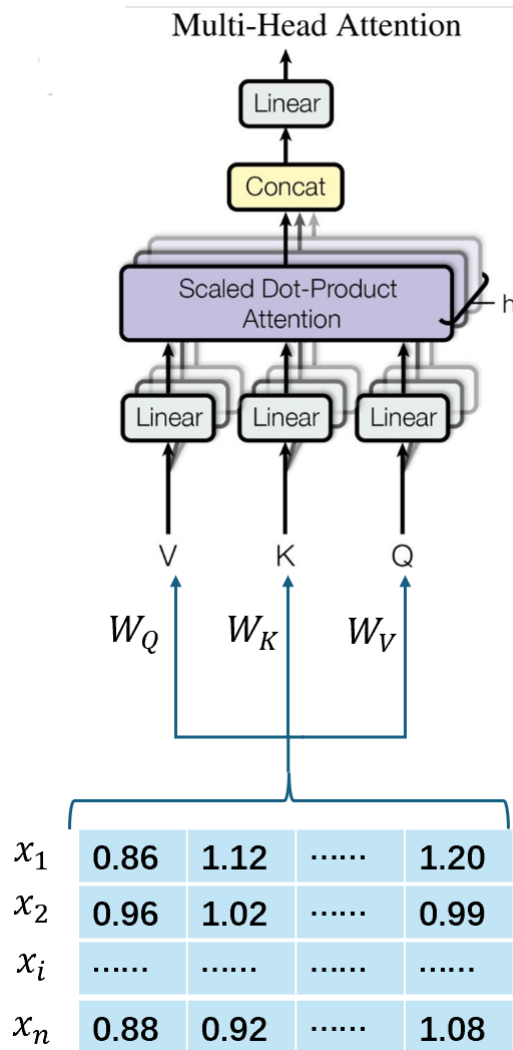
```
print("Attention weights:\n", attention_weights)
```

MHA--多头注意力

单头注意力中，模型只能通过一个注意力头来捕捉输入数据中的特征，这限制了模型对复杂关系的建模能力。而多头注意力（Multi-Head Attention）是Transformer架构的核心组件，它通过将输入数据分解为多个“头”（heads），分别计算注意力，从而能够捕捉到输入数据中不同子空间的特征。并且对比传统单头注意力，其复杂度并没有增加。这种机制极大地提升了模型对复杂关系的建模能力，广泛应用于自然语言处理（NLP）和计算机视觉（CV）等领域。

原理介绍

多头注意力的核心思想是将输入数据分解为多个子空间，每个子空间通过一个独立的注意力头进行处理，最后将所有头的输出合并起来。相关原理图如下所示：



(1) 计算 Query, Key, Value 矩阵

每个输入词都会被映射成三个不同的向量：

- 是查询（Query），其表示当前需要关注的内容，例如在机器翻译中，查询可能是目标语言句子中的一个词。
- 是键（Key），表示与查询进行匹配的内容，例如源语言句子中的词。
- 是值（Value），表示最终要提取的信息，通常与键对应。

定义转换矩阵：

其中，是可学习的参数矩阵。可以发现，获取、的步骤与前面单头注意力是一样的。

(2) 分割多个头

将变换后的查询、键和值分别分割成多个头。假设我们有 个头，每个头的维度为，则有：

其中，是模型的嵌入维度。

分割后的查询、键和值分别如下，其中，表示第 个头。

(3) 计算每个头的注意力

对于每个头，分别计算注意力分数。具体步骤如下：

- 1. 计算点积注意力分数：
- 2. 缩放：缩放因子 的作用是防止点积结果过大导致梯度消失。
- 3. Softmax：将缩放后的分数转换为概率分布。
- 4. 加权求和：

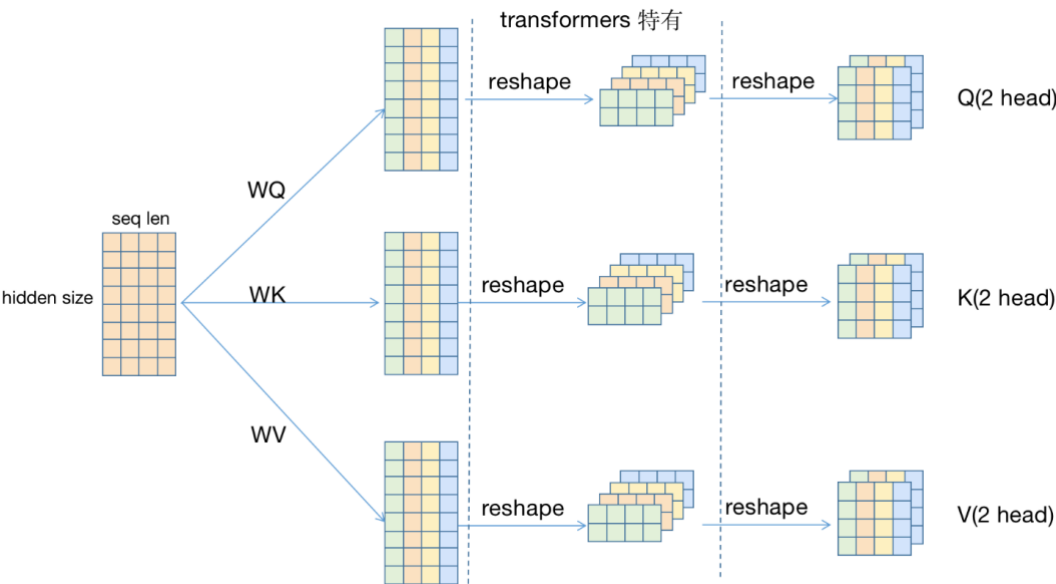
(4) 合并所有头的输出

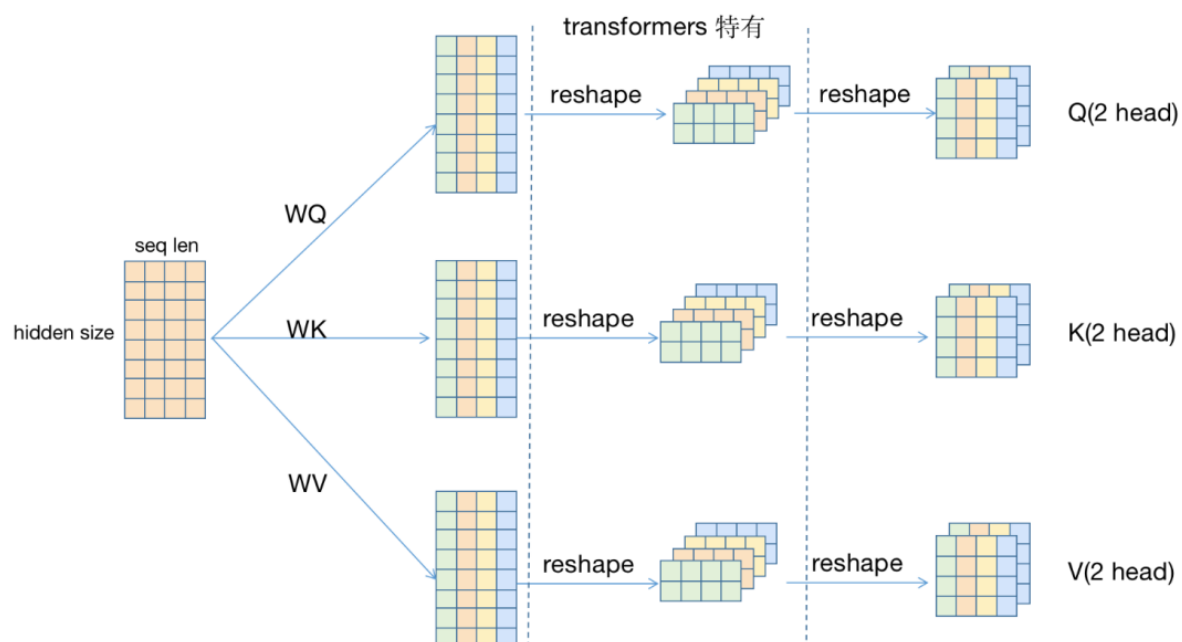
将所有头的输出合并起来，得到最终的输出。合并后的输出再通过一个线性变换：

其中，是另一个可学习的权重矩阵，用于将合并后的输出映射回原始维度。

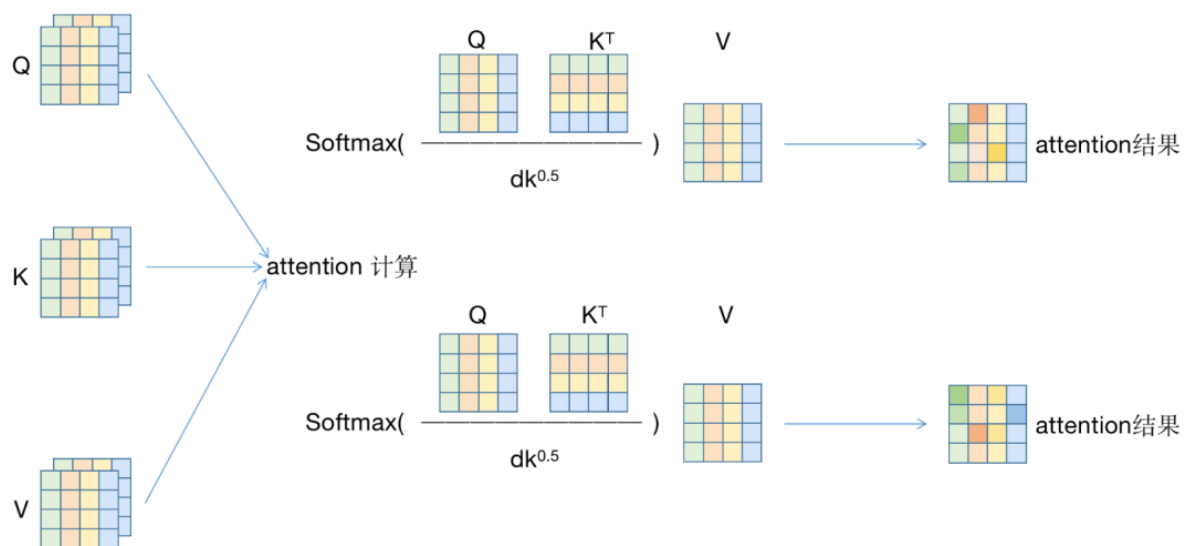
图文理解

1、首先假设一个输入，该输入seq len为4，hidden_size的维度为8，使用2头注意力；同时弱化batch size（假设为1并且不在维度上体现）。如下图所示：

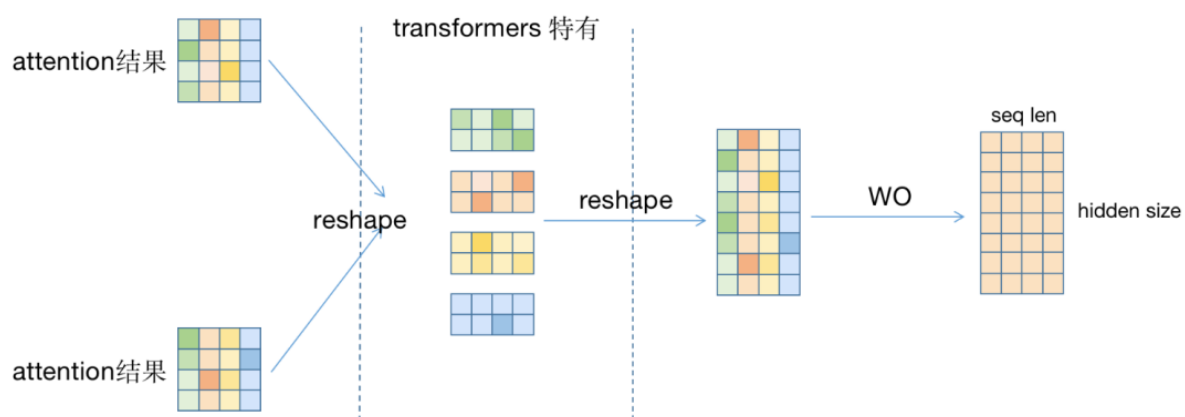




2、对于已经计算得到的QKV，分别计算attention，最终得到了attention的结果，一个矩阵（）



3、获取到了attention的结果后，再经过变换，重新拼接回一个（）矩阵。得到拼接后的8*4矩阵后，经过，得到矩阵，即输出。



示例代码

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class MultiHeadAttention(nn.Module):
    def __init__(self, embed_dim, num_heads):
        """
        多头注意力机制的初始化。
        :param embed_dim: 嵌入维度
        :param num_heads: 头的数量
        """
        super(MultiHeadAttention, self).__init__()
        self.embed_dim = embed_dim
        self.num_heads = num_heads
        self.head_dim = embed_dim // num_heads

        assert self.head_dim * num_heads == embed_dim, "Embed size needs to be divisible by heads"

        # 定义线性层，将输入映射到 Query、Key 和 Value
        self.query_linear = nn.Linear(embed_dim, embed_dim)
        self.key_linear = nn.Linear(embed_dim, embed_dim)
        self.value_linear = nn.Linear(embed_dim, embed_dim)

        # 定义输出的线性层
        self.out = nn.Linear(embed_dim, embed_dim)

        # 缩放因子
        self.scale = torch.sqrt(torch.FloatTensor([self.head_dim]))

    def forward(self, query, key, value):
        """
        多头注意力的前向传播。
        :param query: 查询张量，形状为 [batch_size, seq_len_q, embed_dim]
        :param key: 键张量，形状为 [batch_size, seq_len_k, embed_dim]
        :param value: 值张量，形状为 [batch_size, seq_len_k, embed_dim]
        :return: 输出张量，形状为 [batch_size, seq_len_q, embed_dim]
        """
        batch_size = query.shape[0]

        # 将输入映射到 Query、Key 和 Value
        Q = self.query_linear(query) # [batch_size, seq_len_q, embed_dim]
        K = self.key_linear(key)     # [batch_size, seq_len_k, embed_dim]
        V = self.value_linear(value) # [batch_size, seq_len_k, embed_dim]

        # 分割成多个头
        Q = Q.view(batch_size, -1, self.num_heads, self.head_dim).transpose(1, 2)
        # [batch_size, num_heads, seq_len_q, head_dim]
        K = K.view(batch_size, -1, self.num_heads, self.head_dim).transpose(1, 2)
        # [batch_size, num_heads, seq_len_k, head_dim]
        V = V.view(batch_size, -1, self.num_heads, self.head_dim).transpose(1, 2)
        # [batch_size, num_heads, seq_len_k, head_dim]
```

```

        # 计算点积注意力分数
        attention_scores = torch.matmul(Q, K.transpose(-2, -1)) / self.scale #
        [batch_size, num_heads, seq_len_q, seq_len_k]

        # 应用 Softmax 函数，得到注意力权重
        attention_weights = F.softmax(attention_scores, dim=-1) # [batch_size,
        num_heads, seq_len_q, seq_len_k]

        # 加权求和，得到每个头的输出
        output = torch.matmul(attention_weights, V) # [batch_size, num_heads,
        seq_len_q, head_dim]

        # 合并所有头的输出
        output = output.transpose(1, 2).contiguous().view(batch_size, -1,
        self.embed_dim) # [batch_size, seq_len_q, embed_dim]

        # 通过输出的线性层
        output = self.out(output) # [batch_size, seq_len_q, embed_dim]

        return output, attention_weights

# 示例输入
batch_size = 2
seq_len_q = 3
seq_len_k = 4
embed_dim = 16
num_heads = 4

# 随机生成输入数据
query = torch.randn(batch_size, seq_len_q, embed_dim)
key = torch.randn(batch_size, seq_len_k, embed_dim)
value = torch.randn(batch_size, seq_len_k, embed_dim)

# 初始化多头注意力模块
attention = MultiHeadAttention(embed_dim, num_heads)

# 前向传播
output, attention_weights = attention(query, key, value)

# 打印输出
print("Query:\n", query)
print("Key:\n", key)
print("Value:\n", value)
print("Output:\n", output)
print("Attention weights:\n", attention_weights)

```

承上启下--KV Cache

原理介绍

大模型在解码基本上都是通过自回归的方式进行。即：最新的Token输出依赖于先前生成或者预先填入的Token。举个例子，假如我们输入“窗前明月光下一句是”，那么模型每次生成一个token，输入输出会是这样（方便起见，默认每个token都是一个字符，其中[BOS]和[EOS]分别是起始符号和终止符号）。

```
step0: 输入=[BOS]窗前明月光下一句是; 输出=疑
step1: 输入=[BOS]窗前明月光下一句是疑; 输出=是
step2: 输入=[BOS]窗前明月光下一句是疑是; 输出=地
step3: 输入=[BOS]窗前明月光下一句是疑是地; 输出=上
step4: 输入=[BOS]窗前明月光下一句是疑是地上; 输出=霜
step5: 输入=[BOS]窗前明月光下一句是疑是地上霜; 输出=[EOS]
```

仔细想一下，在生成“疑”字的时候，用的是输入序列中“是”字的最后一层hidden state，通过最后的分类头预测出来的。以此类推，后面每生成一个字，使用的都是输入序列中最后一个字的输出。可以注意到，下一个step的输入其实包含了上一个step的内容，而且只在最后面多了一点点（一个token）。那么下一个step的计算应该也包含了上一个step的计算。从公式上来看是这样的：

1、Attention 计算公式：

注意对于decoder的时候，由于mask attention的存在，每个输入只能看到自己和前面的内容，而看不到后面的内容。

2、假设当前输入的长度是3，预测第4个字，那每层attention所做的计算有

3、预测完第4个字，放到输入里，继续预测第5个字，每层attention所做的计算有：

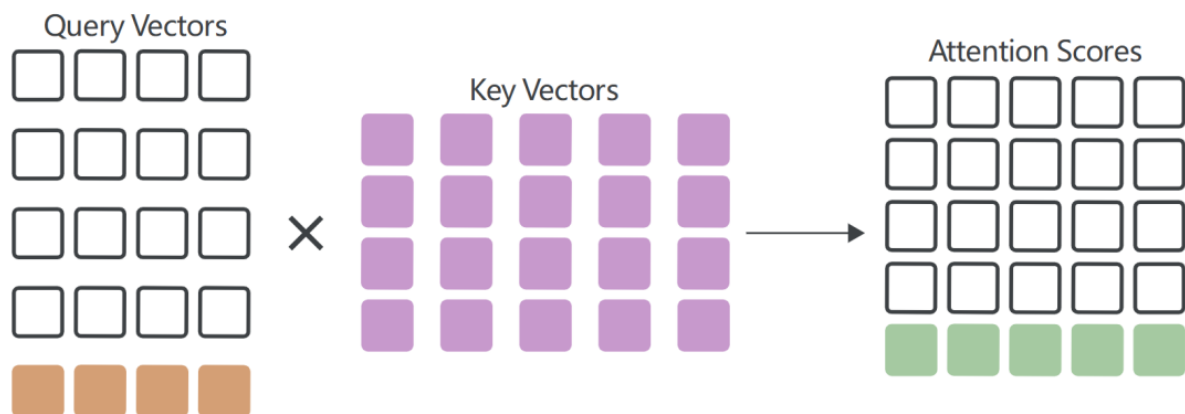
可以看到，在预测第5个字时，只有最后一步引入了新的计算，而 o0 到 o2 的计算和前面是完全重复的。但是模型在推理的时候可不管这些，无论你是不是只要最后一个字的输出，它把所有输入计算一遍，给出所有输出结果。也就是说中间有很多我们用不到的计算，这样就造成了浪费。

图文理解

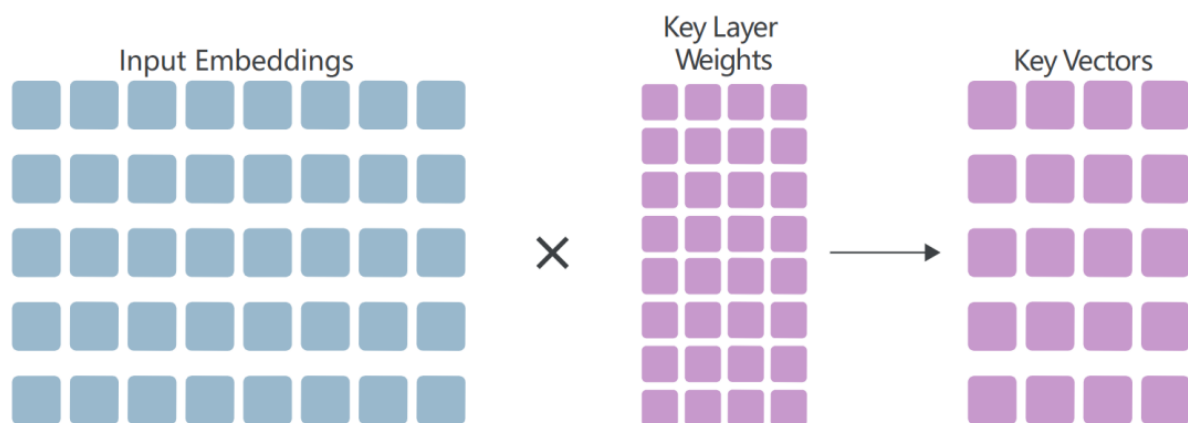
公式理解不了没关系，再来个图文理解。由于decoder是causal的（即，一个token的注意力attention只依赖于它前面的token），在每一步生成过程中，我们实际上是在重复计算相同的前一个token的注意力，而我们真正需要做的是仅计算新token的注意力。这就是KV cache发挥作用的地方。

通过缓存之前的和，我们可以专注于只计算新token的注意力。以下是每个Token的Attention分数的计算过程，可以发现，相比，之前的Attention score是不变的，那么一个新的token进来，只需要计算当前token对应的kv就可以了，后面直接拼起来就好了。这里能做kv cache的主要原因是由于mask矩阵的作用，这就是causal模型的先天优势。

Key向量缓存 我们可以像下面这样将其形象化，其中空的方块代表我们可以从以前的迭代中重用的计算部分：

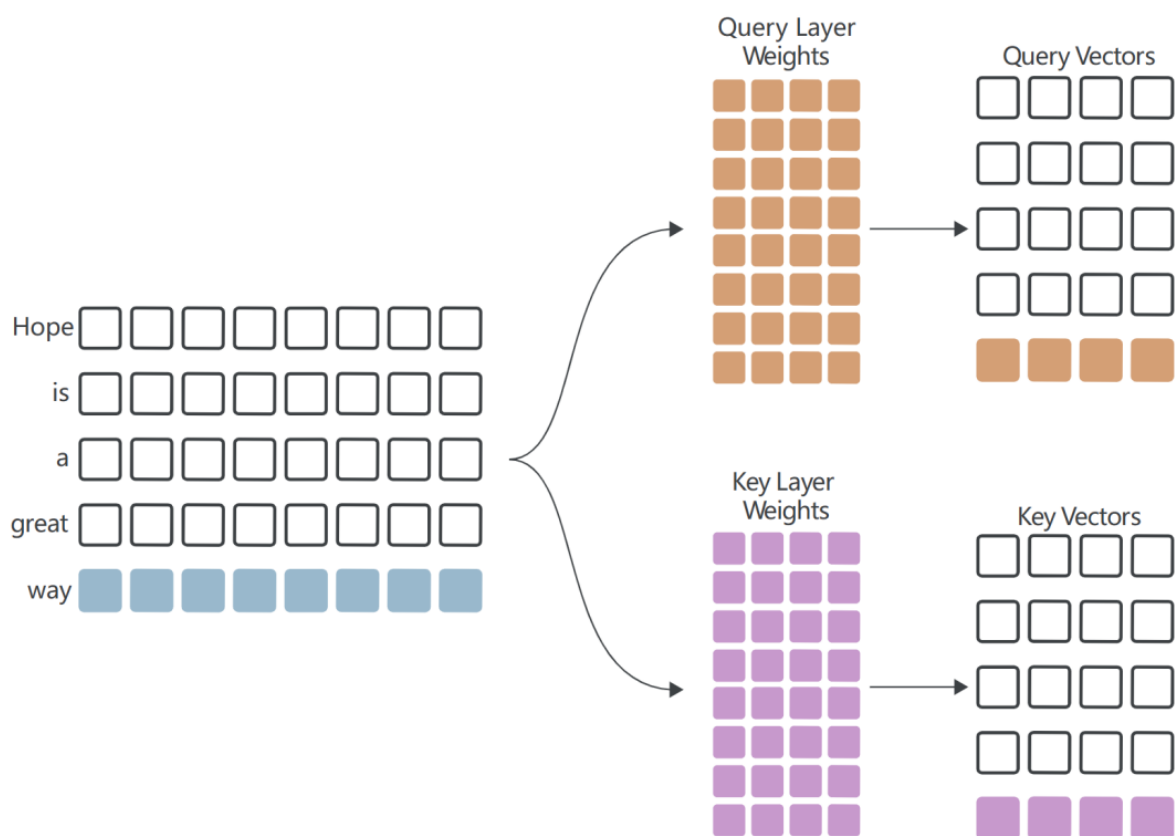


由此我们可以看出，只需要最后一个查询向量和所有关键向量即可计算注意力得分矩阵的最后一行。关键向量本身是通过将输入嵌入乘以关键层权重来计算的，正如我们之前所见：

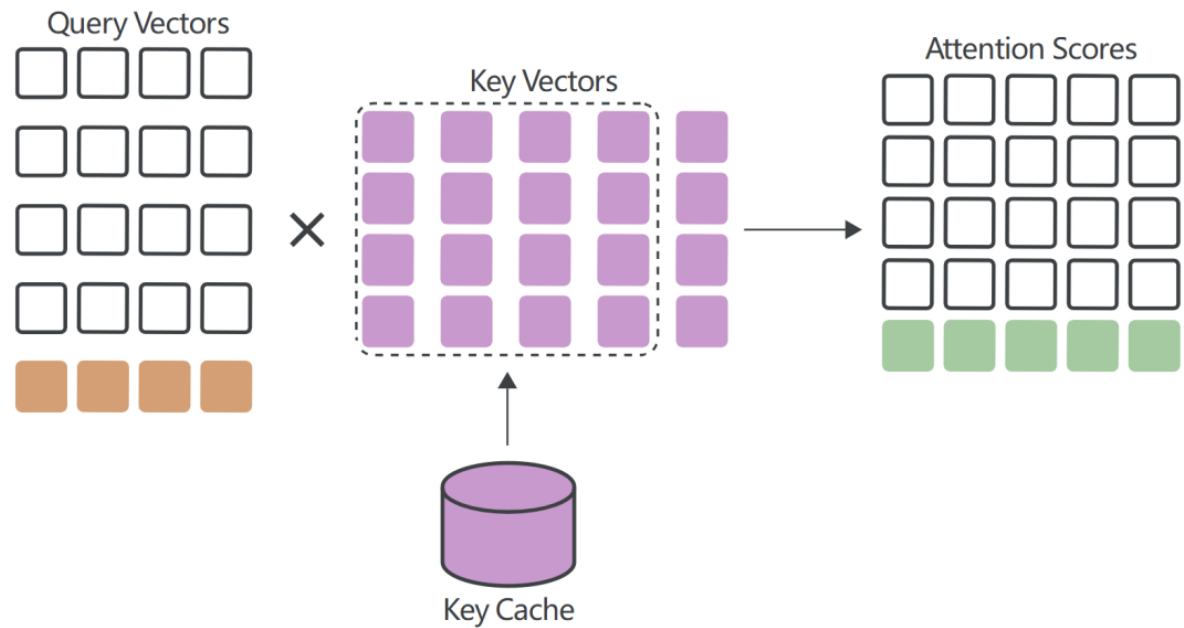


因此，在每次迭代中，我们只需要计算最后一个键向量（因为这取决于最新标记的输入嵌入），而所有其他键向量都可以从上一次迭代中重复使用。

我们可以通过维护一个密钥缓存来节省大量冗余计算，该缓存存储在每次迭代中计算的键向量。首先，我们只计算一个查询向量和一个键向量：

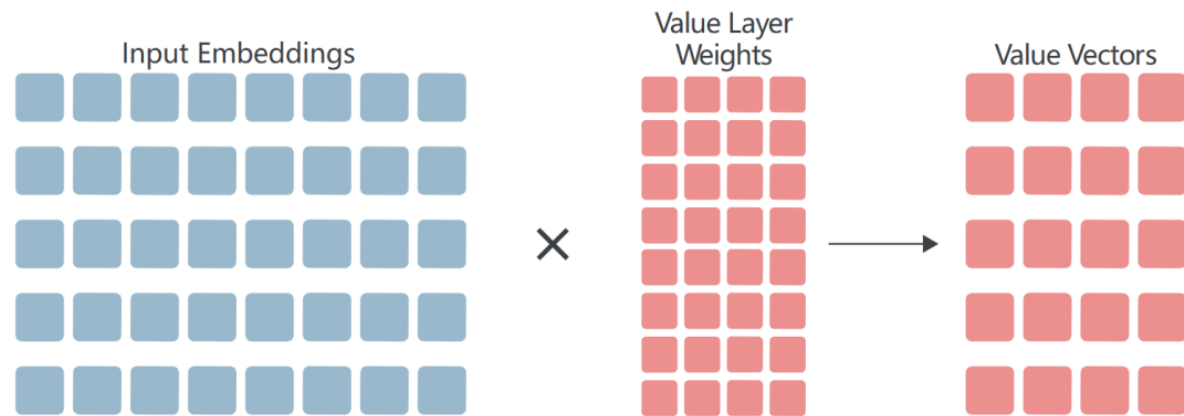


然后，我们从密钥缓存中提取先前计算的密钥向量，并计算注意力分数矩阵的最后一行作为新查询向量与每个密钥向量的点积：

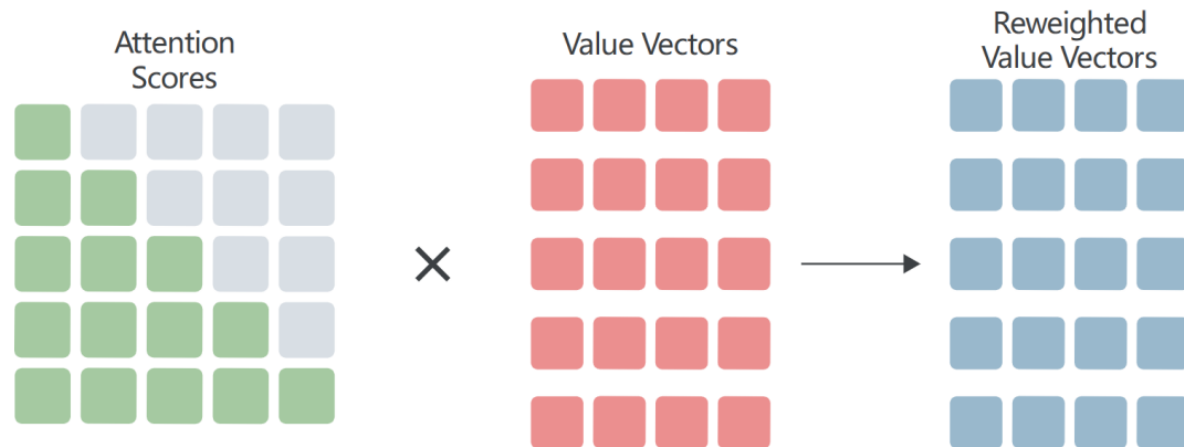


到目前为止，我们已经了解了缓存关键向量如何消除文本生成循环每次迭代中冗余的注意力得分计算。多头自注意力中的值向量也可以在每次迭代中缓存。

回想一下，值向量是通过将输入嵌入发送到第三个线性层来计算的：

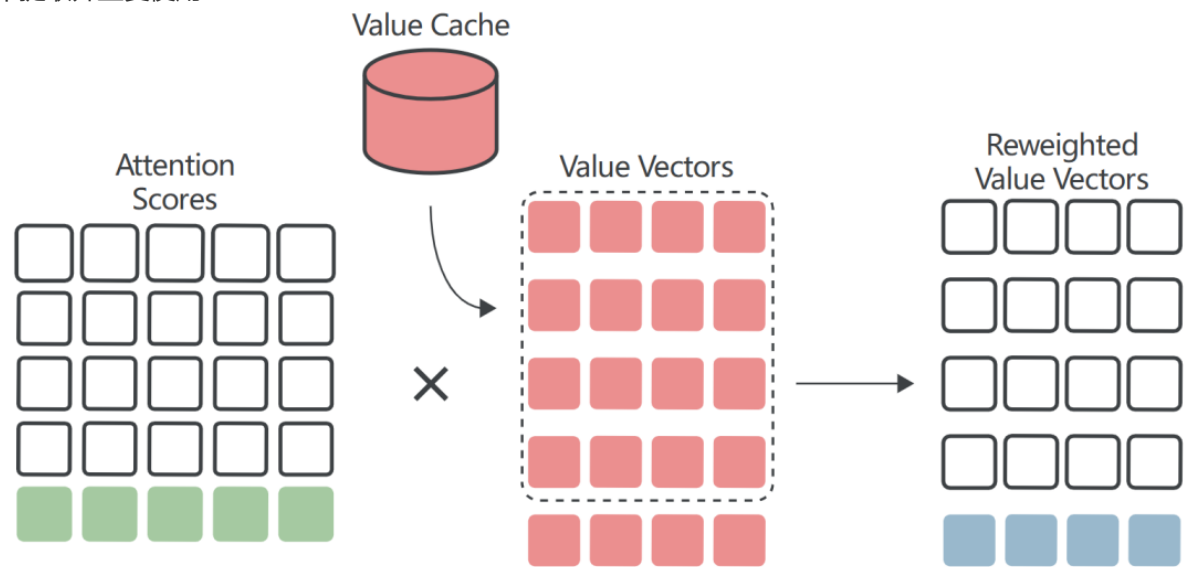


然后，我们通过将两个矩阵相乘来根据注意力得分“重新加权”价值向量：



与键向量一样，每次迭代时只需要计算最后一个（即最新的）值向量。所有其他值向量都可以从值缓存

中提取并重复使用：



实际例子

假设我们的 (Q, K, V) 分别如下：

在计算完 得到 attention 矩阵后，我们创建一个 **masking** 矩阵（其中一个极小值）， 将其与 attention 矩阵相加：

接下来，沿行应用 **softmax**，将这些值转换为概率分布。将 应用于注意力矩阵后，所有这些极小的值都将变为零：

再来看一下不存储 的情况，仅存储 的情况：

相加后的结果与存储 时 **masking** 的结果相同：

应用：

可以观察到，与存储时的结果是一致的，这也代表在接下与矩阵计算得到的 Attention 结果也将一样，这也就是为什么我们在 KV Cache 时不需要存储的原因。

KV Cache 在哪里使用？

当每生成一个新的 token 就会把这个新的 token **append** 进之前的序列中，在将这个序列当作新的输入进行新的 token 生成，直到结束。这使得每次新序列输入时都需要取重复计算前面的 token 的，浪费了很多资源，KV Cache 就是在这里使用的，我们在每次处理新的序列时，可以同时将之前计算的 key, value 一同缓存，并传入下一次计算，这样就节省了很多计算的时间，避免了冗余计算。

KV Cache 节省哪部分内容？

首先，我们要知道，Self-Attention 通过将输入序列变换成三个向量来操作：查询向量（Query），键向量（Key）和值向量（Value）。这些向量是通过将输入进行线性变换得到的。注意力机制基于向量和向量之间的相似度来计算向量的加权求和。然后，将这个加权求和的结果连同原始输入一起送入前馈神经网络，以产生最终输出。

这一过程允许模型专注于相关信息并捕捉长距离依赖关系。那么回到问题，它节省了哪部分计算呢？它节省了对于键（Key）和值（Value）的重复计算，不需要对之前已经计算过的 Token 的和重新进行计算。因为对于之前的 Token 可以复用上一轮计算的结果，避免了重复计算，只需要计算当前 Token 的、。

MQA--多请求注意力

通过KV Cache虽然可以解决kv重复计算的问题，但当面对长上下文的时候，占用的显存是非常可观的。就拿llama3-8B模型来举例子，其模型序列长度 $L=8192$ （即8K），Transformer层数 $N=32$ ，注意力头数 $H=32$ ，每个注意力头的维度 $D=128$ ，Batch按照1来计算，数据类型为BF16（2个字节），那么它所需要的缓存就为：

换算成GB，则为4GB。为了解决这个问题，减少KV缓存一个最直观的方法。

那么MQA多注意力和GQA多注意力应运而生。那么这里首先开始从MQA讲起，多请求注意力（Multi-Query Attention，MQA）是多头注意力（Multi-Head Attention，MHA）的一种变体，由Google团队在2019年提出，其核心：**旨在减少计算开销和显存占用，同时保持一定的模型性能。**

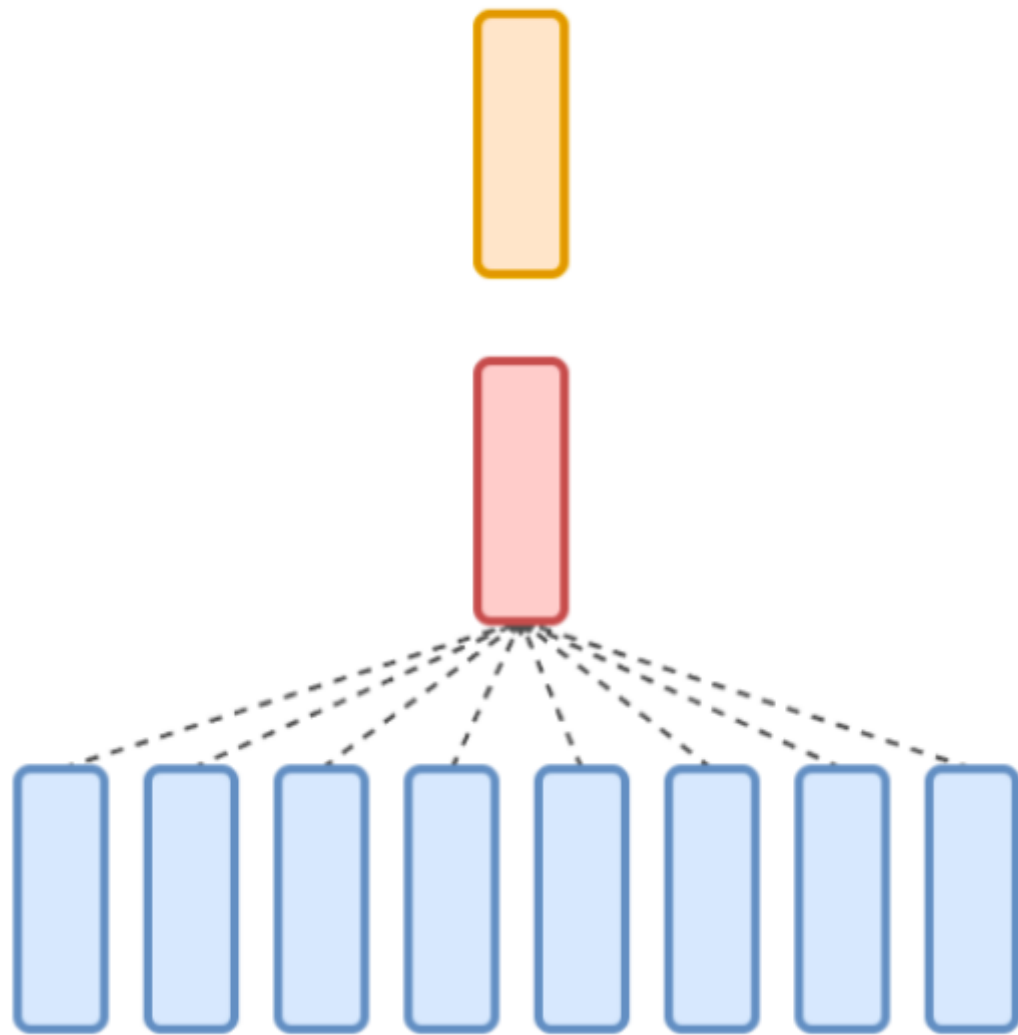
在传统的MHA中，每个注意力头都有独立的查询（Query）、键（Key）和值（Value）矩阵，这使得每个头可以独立学习输入中的不同特性。而**MQA的核心思想是让所有注意力头共享同一份Key和Value矩阵，仅保留Query的多头性质**。这意味着在MQA中，Key和Value的计算是唯一的，而Query则根据不同的头进行独立转换。其优点是：1) KV Cache 显著减少，适合长序列推理；2) 减少了计算和通信开销，推理速度提升 40-50%；缺点是：共享 K 和 V 可能导致模型捕捉上下文的能力下降，任务效果略有损失

原理介绍

MQA的核心思想是**减少Key和Value矩阵的数量**，从而降低计算和存储开销。在传统的MHA中，每个注意力头都有独立的**Query (Q)**、**Key (K)** 和**Value (V)** 矩阵。

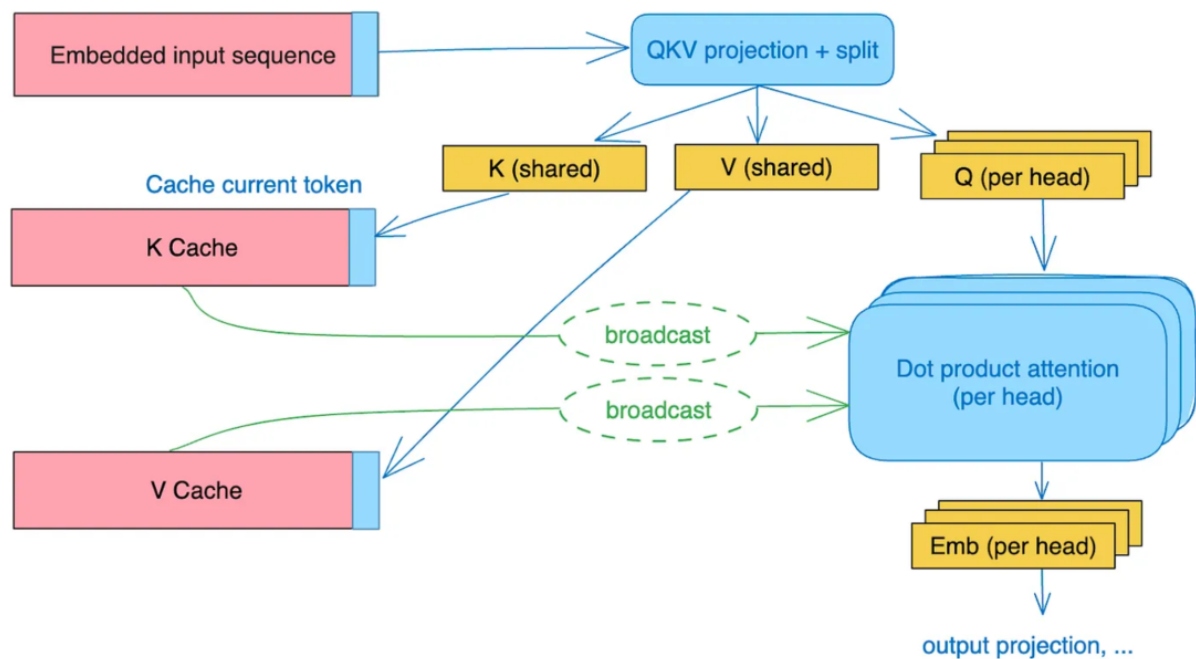
MQA的做法其实很简单。在MHA中，输入分别经过、的变换之后，都切成了n份（n=头数），维度也从降到了，分别进行attention计算再拼接。而MQA这里，在线性变换之后，只对进行切分（和MHA一样），而、则直接在线性变换的时候把维度降到了（而不是切分变小），然后这n个Query头分别和同一份、进行attention计算，之后把结果拼接起来。其经典原理图如下所示：

Multi-query



简单来

说，就是MHA中，每个注意力头的、是不一样的，而MQA这里，每个注意力头的、是一样的，值是共享的。而其他步骤都和MHA一样，具体计算原理图如下：



假设输入为 `hidden_states`，其维度为 `(batch_size, sequence_length, hidden_size)`，以下是MQA的详细计算过程：

1.线性变换:

Query: 每个头的Query矩阵独立计算:

其中, W_Q 是一个线性变换矩阵, 维度为 $(d_{\text{model}}, d_{\text{model}})$ 。

Key: 所有头共享同一个Key矩阵:

其中, W_K 是一个线性变换矩阵, 维度为 $(d_{\text{model}}, d_{\text{model}})$ 。

Value: 所有头共享同一个Value矩阵: 其中, W_V 是一个线性变换矩阵, 维度为 $(d_{\text{model}}, d_{\text{model}})$ 。

2.多头切分

Query: 将Query矩阵按头进行切分:

Key和Value: 由于Key和Value是共享的, 它们不需要按头切分, 但需要扩展维度以匹配Query的维度:

3.注意力计算:

计算Query和Key之间的点积:

应用Softmax函数获取注意力权重: 使用注意力权重对Value进行加权求和:

4.多头合并:

将多头的输出合并为一个矩阵:

通过一个线性变换矩阵 将合并后的矩阵映射到输出维度:

示例代码

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class MultiQueryAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super(MultiQueryAttention, self).__init__()
        self.num_heads = num_heads
        self.d_model = d_model
        self.head_dim = d_model // num_heads

        assert (
            self.head_dim * num_heads == d_model
        )
```

```

    ), "d_model must be divisible by num_heads"

    self.query_linear = nn.Linear(d_model, d_model)
    self.key_linear = nn.Linear(d_model, self.head_dim)
    self.value_linear = nn.Linear(d_model, self.head_dim)
    self.out_linear = nn.Linear(d_model, d_model)

    def forward(self, queries, keys, values, mask=None):
        batch_size = queries.size(0)

        # 线性变换
        Q = self.query_linear(queries) # (batch_size, seq_len, d_model)
        K = self.key_linear(keys)       # (batch_size, seq_len, head_dim)
        V = self.value_linear(values)   # (batch_size, seq_len, head_dim)

        # 分割为多个头
        Q = Q.view(batch_size, -1, self.num_heads, self.head_dim).transpose(1, 2)
        # (batch_size, num_heads, seq_len, head_dim)
        K = K.unsqueeze(1).expand(-1, self.num_heads, -1, -1)
        # (batch_size, num_heads, seq_len, head_dim)
        V = V.unsqueeze(1).expand(-1, self.num_heads, -1, -1)
        # (batch_size, num_heads, seq_len, head_dim)

        # 计算注意力得分
        scores = torch.matmul(Q, K.transpose(-2, -1)) /
        torch.sqrt(torch.tensor(self.head_dim, dtype=torch.float32))
        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)
        attn = F.softmax(scores, dim=-1)
        # 计算注意力输出
        output = torch.matmul(attn, V) # (batch_size, num_heads, seq_len,
        head_dim)

        output = output.transpose(1, 2).contiguous().view(batch_size, -1,
        self.d_model) # (batch_size, seq_len, d_model)

        return self.out_linear(output)

batch_size = 1
seq_len = 3
d_model = 4
num_heads = 2

# 随机生成输入张量
queries = torch.rand(batch_size, seq_len, d_model)
keys = torch.rand(batch_size, seq_len, d_model)
values = torch.rand(batch_size, seq_len, d_model)

# 初始化 MQA 模型
mqa = MultiQueryAttention(d_model, num_heads)

mask = torch.tril(torch.ones(seq_len, seq_len)).unsqueeze(0).unsqueeze(0) # (1,
1, seq_len, seq_len)
print('mask:', mask)
# 前向传播

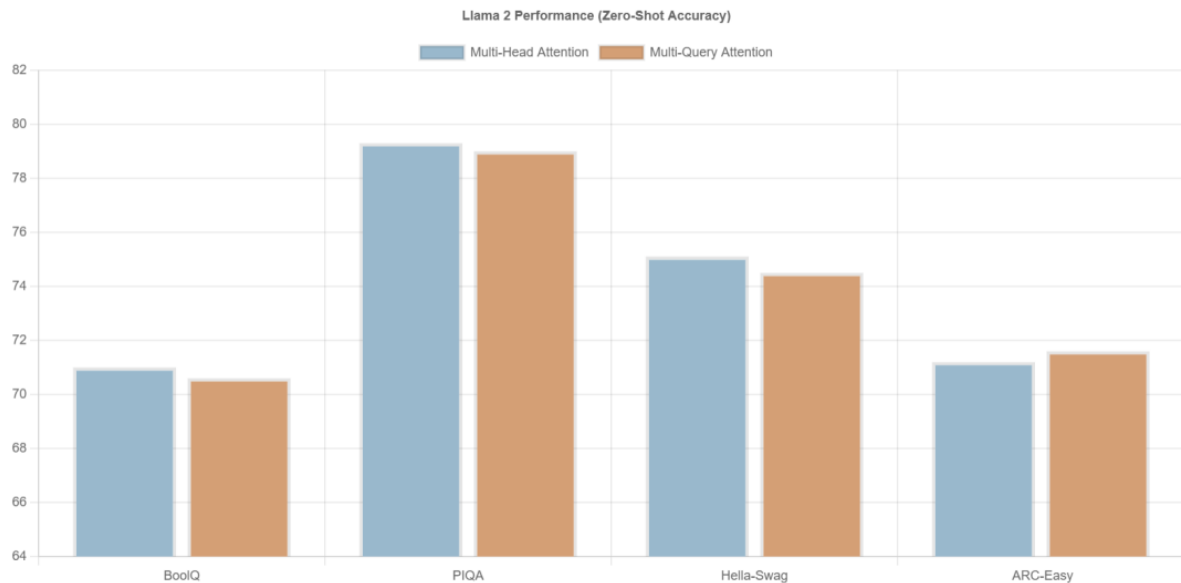
```

```
output = mqa(queries, keys, values,mask)
```

```
print("输出张量: ")  
print(output)
```

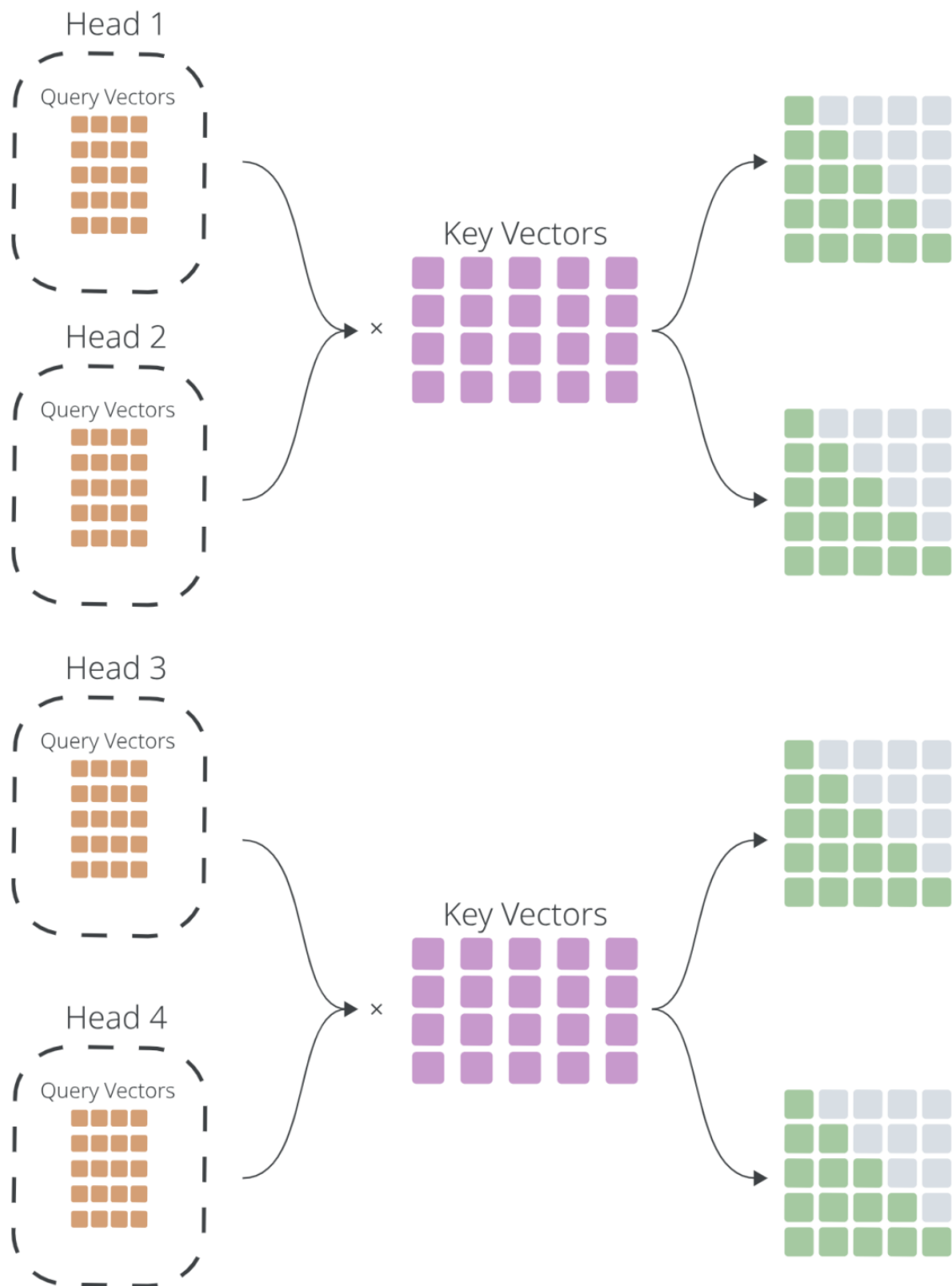
GQA--分组请求注意力

MHA为每个注意力头分配独立的查询、键和值矩阵，增强模型的表达能力，但也增加了计算和内存开销。MQA则让所有注意力头共享同一组键和值矩阵，显著降低了计算成本，但可能影响模型性能。GQA作为折衷方案，通过将查询向量分组，每组共享一组键和值矩阵，旨在在计算效率和模型性能之间取得平衡。在多头注意力（MHA）中，唯一键和值向量的数量等于注意力头的数量；在多查询注意力（MQA）中，唯一键和值向量的数量等于1。



在分组请求注意力（GQA）中，唯一键和值向量的数量等于超参数 G ，即组的数量。例如，如果注意力

头的数量为 4，且 $G=2$ ，那么将有两组唯一的键和价值向量，每组将由两个注意力头使用。



原理介绍

GQA 的目标是减少自注意力计算的复杂度，同时保持 Transformer 的表达能力。它的核心改进点在于：让 **多个 Query 共享少量的 Key 和 Value**，减少计算开销，并通过 **分组机制 (Grouping Mechanism)** 进行更高效的计算，如上图所示。除此之外，为了方便理解 MHA、MQA、MGA 这里再放一个最最经典的图。

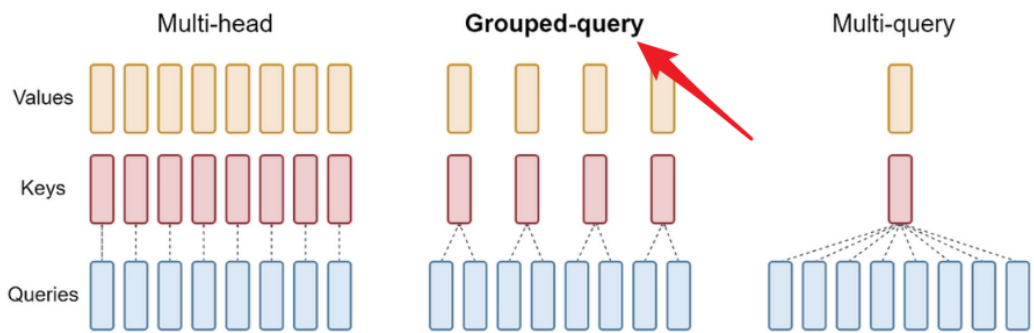


Figure 2: Overview of grouped-query method. Multi-head attention has H query, key, and value heads. Multi-query attention shares single key and value heads across all query heads. Grouped-query attention instead shares single key and value heads for each *group* of query heads, interpolating between multi-head and multi-query attention.

在 GQA 中，Query 仍然是独立计算的，每个 Query 有自己的投影。但 Key 和 Value 是 **共享的**，它们被分组并被多个 Query 使用。GQA 计算过程如下：

1、嵌入向量输入

假设输入序列为：

其中：

- 是序列长度 (tokens 数)
- 是隐藏层维度 (embedding 维度)

在标准自注意力 (Self-Attention) 中，输入 会被投影到 Query、Key、Value 三个子空间：

其中 是可训练的投影矩阵。

2、Query 多头计算

GQA 采用了**分组键值 (Grouped Key-Value, GKV)**，即：

- **Query** (查询) 仍然是**每个头独立计算的**。
- **Key 和 Value 共享**，即 Key 和 Value 只计算**少量的分组**，然后多个 Query 共享这些值。

假设我们有个注意力头 (Heads)，Query 的计算方式如下：

其中：

- 是第 i 个头的 Query 投影矩阵。
- 计算出的 形状为 (L_i, d_q) ，其中 L_i 是第 i 个头的序列长度， d_q 是 Query 的维度。

3、Key 和 Value 计算 (共享分组)

在标准的 MHA (Multi-Head Attention) 中，每个头都单独计算 QK^T 和 QV^T 。但在 GQA 中，我们将 Key 和 Value **分成组 (Groups)**，其中 G 是组的数量，即：

其中：

和只计算组 Key 和 Value，而不是组。

计算出的和形状分别为，其中（即 Key 和 Value 维度比 MHA 更少）。

每个组的 Key 和 Value 将被多个 Query 共享。

这样，每个 Query 头不再独立拥有自己的和，而是共享一组 Key-Value，从而降低计算量。

4、计算注意力得分

注意力得分使用**缩放点积注意力（Scaled Dot-Product Attention）**：

其中：

- 来自**每个 Query 头**。
- 来自**共享的 Key 组**。
- 由于被多个共享，这减少了计算成本。
- 计算出的形状为。

5、计算加权 Value

其中：

- 是注意力得分（来自 Query 和 Key 计算）。
- 是共享的 Value 组。
- 计算出的形状仍为。

6、输出计算

所有注意力头计算的结果会被拼接，然后经过最终的线性变换：

其中：

- 是输出投影矩阵，最终得到形状为的输出。

示例代码

```
import torch
import torch.nn as nn

class GQA(nn.Module):
    def __init__(self, d_model, num_heads, num_groups):
        super(GQA, self).__init__()
        assert num_heads % num_groups == 0, "Heads should be evenly divisible by groups"
        self.num_heads = num_heads
        self.num_groups = num_groups
        self.d_model = d_model
```

```

self.d_head = d_model // num_heads
self.d_group = d_model // num_groups # Key-value 分组维度

# Query 仍然是独立的
self.w_q = nn.Linear(d_model, d_model, bias=False)
# key 和 value 共享
self.w_k = nn.Linear(d_model, d_model // num_groups * num_heads,
bias=False)
self.w_v = nn.Linear(d_model, d_model // num_groups * num_heads,
bias=False)
self.w_o = nn.Linear(d_model, d_model, bias=False)

def forward(self, x):
    batch_size, seq_len, _ = x.shape

    # 计算 Query, Key, Value
    Q = self.w_q(x).view(batch_size, seq_len, self.num_heads, self.d_head)
    K = self.w_k(x).view(batch_size, seq_len, self.num_groups, self.d_group)
    V = self.w_v(x).view(batch_size, seq_len, self.num_groups, self.d_group)

    # 计算注意力分数
    attention_scores = torch.einsum("bqhd,bkgd->bhqk", Q, K) / (self.d_group
** 0.5)
    attention_weights = torch.softmax(attention_scores, dim=-1)

    # 计算注意力加权值
    Z = torch.einsum("bhqk,bkgd->bqhd", attention_weights, V)

    # 重新 reshape 并输出
    Z = Z.reshape(batch_size, seq_len, self.d_model)
    return self.w_o(Z)

```

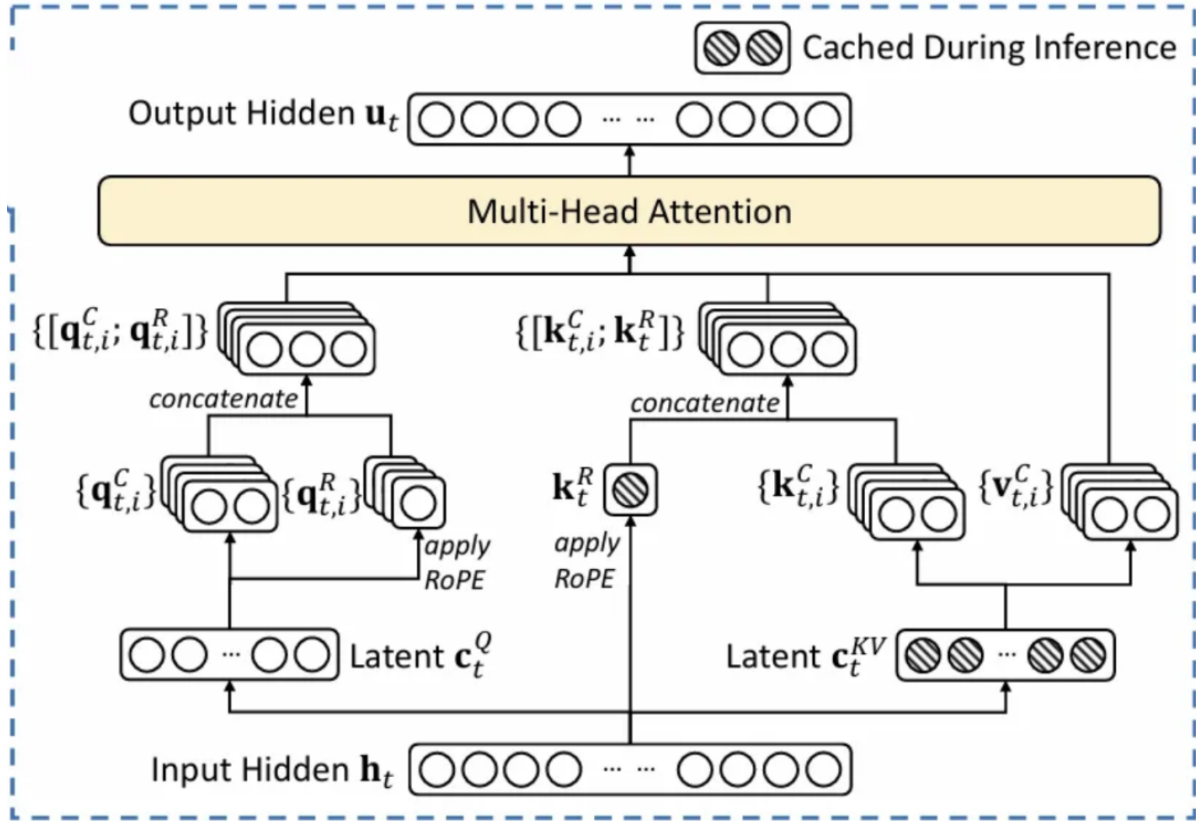
MLA--多潜头注意力

原理介绍

MLA (Multi-Head Local Attention) 的基本思想是将注意力输入 压缩成一个低维的潜在向量，维度为 d_{group} ，其中 $d_{group} \ll d_{model}$ 。在需要计算注意力时，可以将这个潜在向量映射回高维空间，从而恢复键 (keys) 和值 (values)。因此，只需要存储潜在向量，从而显著减少了内存的占用。

先看以下DS得MLA的流程图：

Multi-Head Latent Attention (MLA)



再看一下公式表示：

$$\mathbf{c}_t^Q = W^{DQ} \mathbf{h}_t, \quad (37)$$

$$[\mathbf{q}_{t,1}^C; \mathbf{q}_{t,2}^C; \dots; \mathbf{q}_{t,n_h}^C] = \mathbf{q}_t^C = W^{UQ} \mathbf{c}_t^Q, \quad (38)$$

$$[\mathbf{q}_{t,1}^R; \mathbf{q}_{t,2}^R; \dots; \mathbf{q}_{t,n_h}^R] = \mathbf{q}_t^R = \text{RoPE}(W^{QR} \mathbf{c}_t^Q), \quad (39)$$

$$\mathbf{q}_{t,i} = [\mathbf{q}_{t,i}^C; \mathbf{q}_{t,i}^R], \quad (40)$$

$$\boxed{\mathbf{c}_t^{KV}} = W^{DKV} \mathbf{h}_t, \quad (41)$$

$$[\mathbf{k}_{t,1}^C; \mathbf{k}_{t,2}^C; \dots; \mathbf{k}_{t,n_h}^C] = \mathbf{k}_t^C = W^{UK} \mathbf{c}_t^{KV}, \quad (42)$$

$$\boxed{\mathbf{k}_t^R} = \text{RoPE}(W^{KR} \mathbf{h}_t), \quad (43)$$

$$\mathbf{k}_{t,i} = [\mathbf{k}_{t,i}^C; \mathbf{k}_t^R], \quad (44)$$

$$[\mathbf{v}_{t,1}^C; \mathbf{v}_{t,2}^C; \dots; \mathbf{v}_{t,n_h}^C] = \mathbf{v}_t^C = W^{UV} \mathbf{c}_t^{KV}, \quad (45)$$

$$\mathbf{o}_{t,i} = \sum_{j=1}^t \text{Softmax}_j \left(\frac{\mathbf{q}_{t,i}^T \mathbf{k}_{j,i}}{\sqrt{d_h + d_h^R}} \right) \mathbf{v}_{j,i}^C, \quad (46)$$

$$\mathbf{u}_t = W^O [\mathbf{o}_{t,1}; \mathbf{o}_{t,2}; \dots; \mathbf{o}_{t,n_h}], \quad (47)$$

很懵对不对？仔细看解释来了。

公式 (37-40) 主要是为了计算Q（即Attention中的Q矩阵）。

公式 (37) 通过矩阵对实现降维；

公式 (38) 通过矩阵来实现对的升维，这样一降一升，就大幅降低了本身的权重矩阵参数。

公式 (39) 通过矩阵对进行映射计算，这里在DeepSeek论文中，相当于对又做了一次降维，然后对其做旋转位置编码。（这里为什么要单独做RoPE位置编码呢？可以看后面，为什么需要解耦的RoPE？）

公式（40）一降一升后的，再拼接上旋转位置编码。这样相当于得到了MHA中的Q。

公式（40-45）主要是为了计算K、V（即Attention中的Q矩阵）。

公式（41）通过矩阵对实现降维；

公式（42）通过矩阵对实现升维，得到和前面计算Q一样，通过一降一升，就大幅降低了本身的权重矩阵参数。

公式（43）通过矩阵对进行映射计算，然后对其做RoPE位置编码。（这里和计算Q的旋转位置编码不一样）

公式（44）将的每个头的计算结果分别与RoPE位置编码后的进行拼接得到k，这样相当于得到了MHA中的K。

公式（45）主要是计算矩阵。

公式（46-47）分别计算每个头的注意力，然后拼接到一块，接着利用做个映射，完成Attention计算。

在此过程中，只有上述公式中，带框的蓝色变量需要被缓存，其它的都可以利用“矩阵吸收”，重新恢复过来。

细节介绍

为什么需要解耦的RoPE？

RoPE是训练生成模型以处理长序列的常用选择，简单应用案例如下。如果直接应用上述MLA策略，这将与RoPE不兼容，为什么呢？

$$\begin{bmatrix} x'_i \\ x'_{i+1} \end{bmatrix} = \underbrace{\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}}_{\text{rotation matrix}} \begin{bmatrix} x_i \\ x_{i+1} \end{bmatrix}$$

为了更清楚地理解这一点，考虑当我们使用公式计算注意力时会发生什么：将得转置与相乘时，矩阵和将出现在中间，它们的组合等价于一个从到的单一映射维度。

不加RoPE，我们可以提前计算好，也就上面说的 吸收到 中，这样在做 的变换的时候，也就同时计算了矩阵的乘法。这样的好处是，我们只需要缓存，而不是缓存 的结果。**这就是MLA的压缩KV Cache的核心原理。**

加上RoPE后，为什么不兼容？，计算 乘积，会在 和 之间，增加一个融合了相对位置的变量，如公式所示：

中间这个分量 是随这相对位置变化而变化的，并不是个固定的矩阵，因此并不能提前计算好。所以论文中说RoPE与低秩变换不兼容。（这里对旋转位置编码不熟悉的小伙伴可以重新了解一下旋转位置编码）

怎么解决RoPE不兼容问题呢？ --->通过增加一个很小 分量，引入RoPE

为了引入位置编码，作者在一个很小维度下，用MQA方式计算了，也就是在每层网络中，所有Head只计算一个（如论文中公式43所示）。引入位置编码的向量维度取的比较小为：。

所以最终 向量通过两部分拼接而成，计算权重时，由前后两部分分别相乘再相加得到，如下公式所示：

前一项 按公式 (1) 计算，通过矩阵吸收处理，全Head只缓存一个，后一项 按正常MQA的方式计算，全Head只缓存了一个共享。

通过类似的计算方式，可以处理将 的变换矩阵 吸收到最终的结果变换矩阵 中，这样也不用实际计算和缓存 的值。而是只缓存跟 一样的 即可。

示例代码

```
## DeepSeek MLA源码
class MLA(nn.Module):
    """
    Multi-Headed Attention Layer (MLA).

    Attributes:
        dim (int): Dimensionality of the input features.
        n_heads (int): Number of attention heads.
        n_local_heads (int): Number of local attention heads for distributed
systems.
        q_lora_rank (int): Rank for low-rank query projection.
        kv_lora_rank (int): Rank for low-rank key/value projection.
        qk_nope_head_dim (int): Dimensionality of non-positional query/key
projections.
        qk_rope_head_dim (int): Dimensionality of rotary-positional query/key
projections.
        qk_head_dim (int): Total dimensionality of query/key projections.
        v_head_dim (int): Dimensionality of value projections.
        softmax_scale (float): Scaling factor for softmax in attention
computation.
    """
    def __init__(self, args: ModelArgs):
        super().__init__()
        self.dim = args.dim # 输入特征的维度d
        self.n_heads = args.n_heads # 128
        self.n_local_heads = args.n_heads // world_size # word_size = 1 进程数
        self.q_lora_rank = args.q_lora_rank # 低秩查询投影的秩 0表示不使用低秩 1536
        self.kv_lora_rank = args.kv_lora_rank # 低秩键/值投影的秩 512
        self.qk_nope_head_dim = args.qk_nope_head_dim # 128
        self.qk_rope_head_dim = args.qk_rope_head_dim # 64
        self.qk_head_dim = args.qk_nope_head_dim + args.qk_rope_head_dim #
128+64
        self.v_head_dim = args.v_head_dim # 128

        if self.q_lora_rank == 0:
            self.wq = ColumnParallelLinear(self.dim, self.n_heads *
self.qk_head_dim)
        else:
            self.wq_a = Linear(self.dim, self.q_lora_rank) # q_lora_rank = 1536
7168*1536
            self.q_norm = RMSNorm(self.q_lora_rank) # 1536
            self.wq_b = ColumnParallelLinear(self.q_lora_rank, self.n_heads *
self.qk_head_dim) # 1536 * 128*(128+64)
```

```

        self.wkv_a = Linear(self.dim, self.kv_lora_rank + self.qk_rope_head_dim)
# 7168*(512+64)
        self.kv_norm = RMSNorm(self.kv_lora_rank) # 512
        self.wkv_b = ColumnParallelLinear(self.kv_lora_rank, self.n_heads *
(self.qk_nope_head_dim + self.v_head_dim)) # 512 * 128*(128+128)
        self.wo = RowParallelLinear(self.n_heads * self.v_head_dim, self.dim) #
128*128 * 7168
        self.softmax_scale = self.qk_head_dim ** -0.5
        if args.max_seq_len > args.original_seq_len:
            mscale = 0.1 * args.mscales * math.log(args.rope_factor) + 1.0
            self.softmax_scale = self.softmax_scale * mscale * mscale

        if attn_impl == "naive":
            self.register_buffer("k_cache", torch.zeros(args.max_batch_size,
args.max_seq_len, self.n_local_heads, self.qk_head_dim), persistent=False)
            self.register_buffer("v_cache", torch.zeros(args.max_batch_size,
args.max_seq_len, self.n_local_heads, self.v_head_dim), persistent=False)
        else:
            self.register_buffer("kv_cache", torch.zeros(args.max_batch_size,
args.max_seq_len, self.kv_lora_rank), persistent=False)
            self.register_buffer("pe_cache", torch.zeros(args.max_batch_size,
args.max_seq_len, self.qk_rope_head_dim), persistent=False)

    def forward(self, x: torch.Tensor, start_pos: int, freqs_cis: torch.Tensor,
mask: Optional[torch.Tensor]):
        """
        Forward pass for the Multi-Headed Attention Layer (MLA).

        Args:
            x (torch.Tensor): Input tensor of shape (batch_size, seq_len, dim).
            start_pos (int): Starting position in the sequence for caching.
            freqs_cis (torch.Tensor): Precomputed complex exponential values for
rotary embeddings.
            mask (Optional[torch.Tensor]): Mask tensor to exclude certain
positions from attention.

        Returns:
            torch.Tensor: Output tensor with the same shape as the input.
        """
        bsz, seq_len, _ = x.size() # batch_size, seq_len, dim 1, 2, 7168
        end_pos = start_pos + seq_len
        if self.q_lora_rank == 0:
            q = self.wq(x)
        else:
            q = self.wq_b(self.q_norm(self.wq_a(x))) # 1, 2, 7168 --> 1, 2,
1536 --> 1, 2, 128*(128+64) 先降维，再升维
            q = q.view(bsz, seq_len, self.n_local_heads, self.qk_head_dim) # 1, 2,
128, 128+64
            q_nope, q_pe = torch.split(q, [self.qk_nope_head_dim,
self.qk_rope_head_dim], dim=-1) # 对最后一个维度进行切分 1, 2, 128, 128 && 1, 2, 128,
64
            q_pe = apply_rotary_emb(q_pe, freqs_cis)
            kv = self.wkv_a(x) # 1, 2, 7168 --> 1, 2, 512+64 ( ) # 直接降维
            kv, k_pe = torch.split(kv, [self.kv_lora_rank, self.qk_rope_head_dim],
dim=-1) # 1, 2, 512 && 1, 2, 64
            k_pe = apply_rotary_emb(k_pe.unsqueeze(2), freqs_cis) # 1, 2, 1, 64

```

```

        if attn_impl == "naive":
            q = torch.cat([q_nope, q_pe], dim=-1) # 1, 2, 128, 128+64
            kv = self.wkv_b(self.kv_norm(kv)) # 1, 2, 512 --> 1, 2, 128*(128+128)
对kv进行生维
            kv = kv.view(bsz, seqlen, self.n_local_heads, self.qk_nope_head_dim +
self.v_head_dim) # 1, 2, 128, 128+128 (516)
            k_nope, v = torch.split(kv, [self.qk_nope_head_dim, self.v_head_dim],
dim=-1) # 1, 2, 128, 128 && 1, 2, 128, 128
            k = torch.cat([k_nope, k_pe.expand(-1, -1, self.n_local_heads, -1)],
dim=-1) # 1, 2, 128, 128 && 1, 2, 1, 64 --> 1, 2, 128, 128+64
            self.k_cache[:bsz, start_pos:end_pos] = k
            self.v_cache[:bsz, start_pos:end_pos] = v
            scores = torch.einsum("bshd,bthd->bsht", q, self.k_cache[:bsz,
:end_pos]) * self.softmax_scale # 1, 2, 128, 128 * 1, 2, 128, 128+64
        else:
            wkv_b = self.wkv_b.weight if self.wkv_b.scale is None else
weight_dequant(self.wkv_b.weight, self.wkv_b.scale, block_size) # 512 * 128*
(128+128)
            wkv_b = wkv_b.view(self.n_local_heads, -1, self.kv_lora_rank) # 128,
128+128, 512
            q_nope = torch.einsum("bshd,hdc->bshc", q_nope, wkv_b[:,
:self.qk_nope_head_dim]) # 1, 2, 128, 128
            self.kv_cache[:bsz, start_pos:end_pos] = self.kv_norm(kv) # 1, 2, 512
--> 1, 2, 512
            self.pe_cache[:bsz, start_pos:end_pos] = k_pe.squeeze(2) # 1, 2, 1, 64
--> 1, 2, 64
            scores = (torch.einsum("bshc,btc->bsht", q_nope, self.kv_cache[:bsz,
:end_pos]) +
                    torch.einsum("bshr,btr->bsht", q_pe, self.pe_cache[:bsz,
:end_pos])) * self.softmax_scale # 1, 2, 128, 128 * 1, 2, 128, 128+64
            if mask is not None:
                scores += mask.unsqueeze(1)
            scores = scores.softmax(dim=-1, dtype=torch.float32).type_as(x)
            if attn_impl == "naive":
                x = torch.einsum("bsht,bthd->bshd", scores, self.v_cache[:bsz,
:end_pos]) # 1, 2, 128, 128
            else:
                x = torch.einsum("bsht,btc->bshc", scores, self.kv_cache[:bsz,
:end_pos])
                x = torch.einsum("bshc,hdc->bshd", x, wkv_b[:, -self.v_head_dim:])
            x = self.wo(x.flatten(2)) # 1, 2, 128, 128 --> 1, 2, 7168
        return x

```

MHA、MQA、GQA、MLA实验结果对比

下表比较了 MHA、GQA、MQA 和 MLA 之间每个 token 所需的 KV 缓存元素数量以及建模容量，表明 MLA 确实可以在内存效率和建模容量之间取得更好的平衡。

Attention Mechanism	KV Cache per Token (# Element)	Capability
Multi-Head Attention (MHA)	$2n_h d_h l$	Strong
Grouped-Query Attention (GQA)	$2n_g d_h l$	Moderate
Multi-Query Attention (MQA)	$2d_h l$	Weak
MLA (Ours)	$(d_c + d_h^R)l \approx \frac{9}{2}d_h l$	Stronger

更具体地说，下表展示了 MHA、GQA 和 MQA 在 7B 模型上的表现，其中 MHA 明显优于 MQA 和

GQA。

Benchmark (Metric)	# Shots	Dense 7B w/ MQA	Dense 7B w/ GQA (8 Groups)	Dense 7B w/ MHA
# Params	-	7.1B	6.9B	6.9B
BBH (EM)	3-shot	33.2	35.6	37.0
MMLU (Acc.)	5-shot	37.9	41.2	45.2
C-Eval (Acc.)	5-shot	30.0	37.7	42.9
CMMLU (Acc.)	5-shot	34.6	38.4	43.5

对 MHA 和 MLA 进行了分析，结果总结在下表中，其中 MLA 整体上取得了更好的效果。

Benchmark (Metric)	# Shots	Small MoE w/ MHA	Small MoE w/ MLA	Large MoE w/ MHA	Large MoE w/ MLA
# Activated Params	-	2.5B	2.4B	25.0B	21.5B
# Total Params	-	15.8B	15.7B	250.8B	247.4B
KV Cache per Token (# Element)	-	110.6K	15.6K	860.2K	34.6K
BBH (EM)	3-shot	37.9	39.0	46.6	50.7
MMLU (Acc.)	5-shot	48.7	50.0	57.5	59.0
C-Eval (Acc.)	5-shot	51.6	50.9	57.9	59.2
CMMLU (Acc.)	5-shot	52.3	53.4	60.7	62.5

文中公式表达或有错误，欢迎批评指正！